02 · Speicherhierarchie & Caches

Mikrocomputertechnik 3 · Vom Lokalitätsprinzip bis zur Code-Optimierung

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 82

Willkommen zu Block 2!

- Willkommen zurück zur zweiten Vorlesung in Mikrocomputertechnik 3
- Rückblick: Block 1 hat Speicher auf Transistor-/Kondensatorebene untersucht (Flash, SRAM, DRAM)
- Heute: Bindeglied zwischen schneller CPU und langsamem RAM
- Fokus: das Konzept der Caches
- Ziel: verstehen, warum die Art Ihres C-Codes die Hardware massiv beeinflusst



Notizen

Willkommen zu Block 2 schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Willkommen zurück zur zweiten Vorlesung in Mikrocomputertechnik 3. Rückblick: Block 1 hat Speicher auf Transistor-/Kondensatorebene untersucht (Flash, SRAM, DRAM). Heute: Bindeglied zwischen schneller CPU und langsamem RAM. Fokus: das Konzept der Caches. Ziel: verstehen, warum die Art Ihres C-Codes die Hardware massiv beeinflusst. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 2 / 82

Anknüpfung: Memory Map und Linker

Kurz-Recap aus Block 1 (Labor folgt mit dem Übungsblatt):

- Memory Map: Flash, SRAM und MMIO belegen feste Adressbereiche
- Linker-Skript legt `.text` in den Flash, Variablen in den SRAM
- C-Pointer adressieren physische Bausteine über denselben Adressraum

```
// Prinzip aus Block 1:
int global_var = 42;   // typisch: SRAM (.data)
const int config = 1;  // typisch: Flash (.rodata)
```

Notizen

Anknüpfung: Memory Map und Linker schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Memory Map: Flash, SRAM und MMIO belegen feste Adressbereiche. Linker-Skript legt `.text` in den Flash, Variablen in den SRAM. C-Pointer adressieren physische Bausteine über denselben Adressraum. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 3 / 82

Kurz-Recap: SRAM vs. DRAM

Aus Block 1 (Spickzettel) — die Motivation für Caches:

- SRAM: schnell und flächenintensiv, häufig für Caches. Registerfiles können auch aus Flip-Flops oder Latches bestehen, nicht nur aus 6T-Zellen
- DRAM (1T1C): dicht und günstig, aber hohe Latenz (Refresh, Multiplexing)
- Wunsch „alles SRAM“ ist wirtschaftlich und flächenmäßig unrealistisch

	SRAM	DRAM
Geschwindigkeit	sehr hoch	niedriger
Kosten / Bit	hoch	niedrig
Typische Rolle	Cache	Hauptspeicher

Schematisch — Trendillustration aus Block 1.

Notizen

Kurz-Recap: SRAM vs. DRAM schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. SRAM: schnell und flächenintensiv, häufig für Caches. Registerfiles können auch aus Flip-Flops oder Latches bestehen, nicht nur aus 6T-Zellen. DRAM (1T1C): dicht und günstig, aber hohe Latenz (Refresh, Multiplexing). Wunsch „alles SRAM“ ist wirtschaftlich und flächenmäßig unrealistisch. SRAM DRAM. Geschwindigkeit sehr hoch niedriger. Kosten / Bit hoch niedrig. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 4 / 82

Die CPU hungert — das Latenz-Problem

- Moderne CPU: ca. 3–5 GHz
- DRAM-Latenz: ca. 50–100 ns
- Daraus folgen grob 150–500 Takte, nicht 100–300
- Das einfache blockierende Modell unten nutzt getrennt 100 Takte Wartezeit

Schritt	Dauer (einfaches blockierendes Modell)
ALU Add	1 Takt
Warten auf DRAM	100 Takte in diesem Modell
ALU Sub	1 Takt

Notizen

Die CPU hungert — das Latenz-Problem schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Moderne CPU: ca. 3–5 GHz. DRAM-Latenz: ca. 50–100 ns. Daraus folgen grob 150–500 Takte, nicht 100–300. Das einfache blockierende Modell unten nutzt getrennt 100 Takte Wartezeit. Schritt Dauer (einfaches blockierendes Modell). ALU Add 1 Takt. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 5 / 82

Die Illusion erschaffen

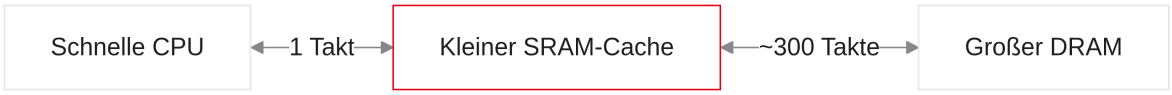
- Ziel: Nutzer und CPU „betrügen“
- Illusion: Speicher so schnell wie SRAM und so billig/groß wie DRAM
- Trick: kleine, versteckte SRAM-Schicht zwischen CPU und DRAM
- Name: Cache (französisch: geheimes Versteck)
- Ideal: CPU merkt kaum, dass DRAM existiert

Notizen

Die Illusion erschaffen schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ziel: Nutzer und CPU „betrügen“. Illusion: Speicher so schnell wie SRAM und so billig/groß wie DRAM. Trick: kleine, versteckte SRAM-Schicht zwischen CPU und DRAM. Name: Cache (französisch: geheimes Versteck). Ideal: CPU merkt kaum, dass DRAM existiert. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 6 / 82

Cache als Zwischenschicht



Notizen

Cache als Zwischenschicht schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 7 / 82

Agenda für den heutigen Tag

1. Memory Wall und Lokalitätsprinzip — warum Caches funktionieren
2. Cache-Grundlagen und Terminologie (Hit, Miss, AMAT)
3. Cache-Mapping I: Direct Mapped
4. Cache-Mapping II: Set-Associative
5. Schreibstrategien: Write-Through vs. Write-Back
6. Brückenschlag zum C-Code; Ausblick Labor (z. B. Ripes)

Notizen

Agenda für den heutigen Tag schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 8 / 82

Lernziele Block 2

- Memory Wall erklären können
- Zeitliche und räumliche Lokalität unterscheiden
- Hit Rate, Miss Penalty und AMAT berechnen
- Direct-Mapped und Set-Associative Caches unterscheiden
- C-Code so schreiben, dass Cache-Hits maximiert werden

Notizen

Lernziele Block 2 schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Memory Wall erklären können. Zeitliche und räumliche Lokalität unterscheiden. Hit Rate, Miss Penalty und AMAT berechnen. Direct-Mapped und Set-Associative Caches unterscheiden. C-Code so schreiben, dass Cache-Hits maximiert werden. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis.

Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 9 / 82

Die Leitfrage des Tages

- Ein Cache ist nutzlos, wenn nicht die benötigten Daten darin liegen
- Hardware muss „ahnen“, was die Software als Nächstes tut
- Problem: Silizium kann keine C-Programme lesen und nicht in die Zukunft sehen
- Leitfrage: Welche Adressmuster nutzt ein Cache ohne das C-Programm zu lesen?
- Eine Hit Rate von 95 % ist ein Rechenbeispiel, keine Garantie und keine Vorhersagegenauigkeit der nächsten Adresse

Notizen

Die Leitfrage des Tages schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ein Cache ist nutzlos, wenn nicht die benötigten Daten darin liegen. Hardware muss „ahnen“, was die Software als Nächstes tut. Problem: Silizium kann keine C-Programme lesen und nicht in die Zukunft sehen. Leitfrage: Welche Adressmuster nutzt ein Cache ohne das C-Programm zu lesen? Eine Hit Rate von 95 % ist ein Rechenbeispiel, keine Garantie und keine Vorhersagegenauigkeit der nächsten Adresse. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 10 / 82

Die Memory Wall im Detail

- Seit ~1980: CPU-Geschwindigkeit wuchs jahrzehntelang stark
- DRAM: deutlich langsameres Wachstum der Latenz/Performance
- Diese öffnende Schere heißt Memory Wall (Speichermauer)
- Ab einem Punkt bringt ein schnellerer Kern kaum noch Zuwachs
- Flaschenhals: Datenlieferung, nicht die ALU

Epoche (schematisch)	CPU relativ	DRAM relativ
1980	$1\times$	$1\times$
2000	$\approx 100\times$	$\approx 4\times$
2020	$\approx 10\,000\times$	$\approx 16\times$

Schematische Trendillustration, workloadabhängig. Keine Messreihe und kein Beweis für 0 % Mehrleistung.

Notizen

Die Memory Wall im Detail schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Seit ~1980: CPU-Geschwindigkeit wuchs jahrzehntelang stark. DRAM: deutlich langsameres Wachstum der Latenz/Performance. Diese öffnende Schere heißt Memory Wall (Speichermauer). Ab einem Punkt bringt ein schnellerer Kern kaum noch Zuwachs. Flaschenhals: Datenlieferung, nicht die ALU. Epoche (schematisch) CPU relativ DRAM relativ. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 11 / 82

Zyklen vs. Zugriffszeiten

Beispiel: RISC-V-SoC mit 3 GHz

- 1 Takt: $\frac{1}{3 \cdot 10^9}$ s $\approx 0{,}33$ ns
- L1-Cache: ca. 1 ns $\rightarrow \approx 3$ Takte
- DRAM: ca. 100 ns $\rightarrow \approx 300$ Takte
- Im einfachen blockierenden Modell kostet ein abhängiger Hauptspeicherzugriff hunderte Takte. Andere Kerne können Arbeit überlappen

Notizen

Bei 3 GHz dauert ein Takt etwa 0,333 ns. Eine angenommene L1-Zeit von 1 ns entspricht etwa drei Taktperioden. 100 ns entsprechen etwa 300 Takten. Die Tabelle mit 100 Takten ist ein getrenntes blockierendes Modell. Nur ein vollständig abhängiger Zugriff wartet so lange. Eine Out-of-Order-CPU kann andere Befehle überlappen. Diese Zahlen beschreiben nicht den Kurs-Mikrocontroller. L1 bedeutet Level 1. Die 3-GHz-Rechnung und das AMAT-Modell mit einem Takt Hit Time plus 100 Takten zusätzlicher Penalty sind zwei getrennte Illustrationen. 300 Takte sind hier die angenommene DRAM-Größenordnung, nicht die Miss Penalty der späteren AMAT-Rechnung. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 12 / 82

Stalls und Bubbles — Link zu MCT1

- Hazard-Logik kann im einfachen blockierenden Modell Fetch und Decode anhalten
- Eine Bubble ist ein eingefügter Leertakt, keine aus dem Speicher geladene NOP-Instruktion
- Unabhängige Befehle können in anderen Implementierungen weiterlaufen

```
lw   t0, 0(a0)      # abhängiger Load, blockierendes Modell
add  t1, t0, t2     # wartet auf t0
```

Notizen

Stalls und Bubbles — Link zu MCT1 schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Hazard-Logik kann im einfachen blockierenden Modell Fetch und Decode anhalten. Eine Bubble ist ein eingefügter Leertakt, keine aus dem Speicher geladene NOP-Instruktion. Unabhängige Befehle können in anderen Implementierungen weiterlaufen. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete

Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 13 / 82

Die Lösung: Speicherhierarchie

- L1: winzig (z. B. 32 KB), on-chip SRAM, 1–3 Takte
- L2: größer (z. B. 512 KB), SRAM, 10–15 Takte
- L3: sehr groß (z. B. 32 MB), SRAM, 30–40 Takte
- Main Memory: riesig (z. B. 16 GB), DRAM, ≈ 300 Takte
- Hardware kopiert Blöcke automatisch zwischen den Schichten

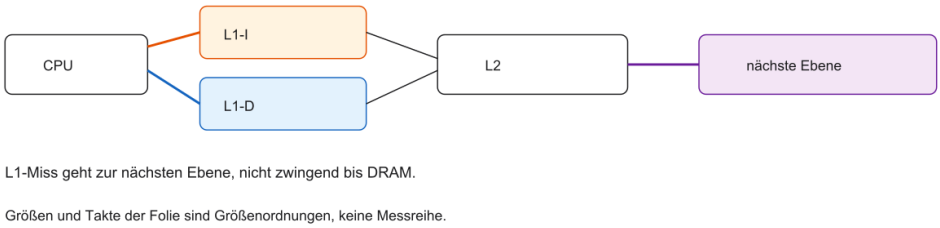


Abbildung 1: Fetch geht zum L1-I, Datenzugriff zum L1-D. Ein L1-Miss fragt die nächste Ebene, nicht zwingend DRAM.

- L1 klein und nah, weitere SRAM-Ebenen größer, DRAM oft die unterste Ebene
- Takte auf der Folie sind Größenordnungen, keine Messreihe

Notizen

Die Lösung: Speicherhierarchie schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. L1: winzig (z. B. 32 KB), on-chip SRAM, 1–3 Takte. L2: größer (z. B. 512 KB), SRAM, 10–15 Takte. L3: sehr groß (z. B. 32 MB), SRAM, 30–40 Takte. Main Memory: riesig (z. B. 16 GB), DRAM, ≈ 300 Takte. Hardware kopiert Blöcke automatisch zwischen den Schichten. L1 klein und nah, weitere SRAM-Ebenen größer, DRAM oft die unterste Ebene. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 14 / 82

Die Geheimwaffe: Lokalitätsprinzip

- Warum funktioniert die Pyramide so gut?
- Software zeigt kein rein zufälliges Zugriffsverhalten
- Code hat Struktur: Schleifen, Arrays
- Principle of Locality (Lokalitätsprinzip)
- Zwei Säulen: zeitliche und räumliche Lokalität

Notizen

Gleichverteilte zufällige Zugriffe haben keine starke Lokalität. Ein endlicher, schon gefüllter Cache kann trotzdem zufällig treffen. Die Trefferquote liegt dann oft niedrig, aber nicht exakt bei null. Gleichverteilung ist ein Modell, kein Synonym für jedes Programm. Zeitliche Lokalität nutzt dieselbe Adresse erneut, räumliche Lokalität die Nachbarbytes derselben ausgerichteten Line. Ein Working Set ist die aktuell häufig benutzte Menge. Der Cache arbeitet auf Adressen, nicht auf C-Variablennamen. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 15 / 82

Säule 1: Zeitliche Lokalität

- Temporal Locality: eine gerade genutzte Adresse wird sehr wahrscheinlich bald wieder genutzt
- Metapher: aufgeschlagenes Buch bleibt auf dem Schreibtisch
- Hardware: geladene Daten bleiben eine Weile im Cache

Notizen

Säule 1: Zeitliche Lokalität schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Temporal Locality: eine gerade genutzte Adresse wird sehr wahrscheinlich bald wieder genutzt. Metapher: aufgeschlagenes Buch bleibt auf dem Schreibtisch. Hardware: geladene Daten bleiben eine Weile im Cache. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 16 / 82

Zeitliche Lokalität in einer Zugriffsspur

Die drei Loads sind Lehrcode, nicht die Übersetzung einer Schleife. `a0` ist hier `0x1000`.

```
# Lehrcode – nicht assembliert oder ausgeführt.
lw t0, 0(a0)    # 0x1000
lw t0, 0(a0)    # wieder 0x1000
lw t1, 4(a0)    # 0x1004
```

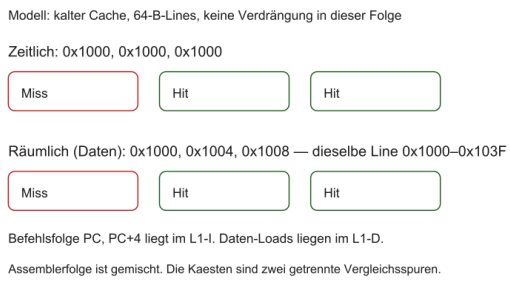


Abbildung 2: Zeitliche Wiederholung derselben Adresse gegen benachbarte Adressen in einer Line.

Notizen

Die drei `lw` sind eine angenommene Zugriffsspur, kein Compilerergebnis. `a0` hält im Modell die ausgerichtete Adresse 0x1000. Der erste Load verfehlt im kalten Cache, der zweite trifft dieselbe Adresse, der dritte liest 0x1004 und liegt in derselben 64-Byte-Line. Die Kästen der Abbildung sind zwei getrennte Vergleichsspuren, nicht diese gemischte Assemblerfolge. `volatile` erzwingt keinen Cache-Bypass. Die nächste Folie betrachtet nur die Nachbaradressen. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt.

Folie 17 / 82

Säule 2: Räumliche Lokalität

- Spatial Locality: nach einer Adresse folgen sehr wahrscheinlich die Nachbarn
- Metapher: Band 2 der Enzyklopädie — Band 3 gleich mitnehmen
- Hardware: nie nur ein Byte — immer ein ganzer Block (Cache Line)

Notizen

Säule 2: Räumliche Lokalität schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte

Blockkopien und verkürzt häufige Zugriffe im Mittel. Spatial Locality: nach einer Adresse folgen sehr wahrscheinlich die Nachbarn. Metapher: Band 2 der Enzyklopädie — Band 3 gleich mitnehmen. Hardware: nie nur ein Byte — immer ein ganzer Block (Cache Line). Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Räumliche Lokalität im C-Code

Zwei typische Quellen:

1. Sequenzielle Befehle: nach Befehl n oft Befehl $n + 1$
2. Arrays und Strings: benachbarte Elemente hintereinander

Wenn `array[0]` geladen wird, lädt der Cache oft schon `array[1]`, `[2]`, ...

```c
/* Lehrcode - nicht kompiliert oder ausgeführt.
   Didaktischer Ausschnitt: Stores, nicht die Load-Spur. */
int daten[100];
for (int i = 0; i < 100; i++) {
    daten[i] = 0;
}
```

Notizen

Die Schleife schreibt Nullen, sie lädt das Array nicht. Das ist ein Store-Beispiel. Ob die Line in den Cache kommt, hängt von Write-Allocate ab. Der Program Counter steigt bei einer 32-Bit-Instruktion ohne Sprung um vier. Komprimierte RISC-V-Instruktionen und Sprünge ändern das. Eine C-Schleife ist ohne Übersetzung keine beobachtete Maschinenspur. Lehrcode, nicht kompiliert. uint32_t belegt hier vier Byte. Ein zusammenhängendes Array im Adressmodell ist keine Zusage über die physische DRAM-Zellgeometrie. Für einen Store-Hit muss die Line vorhanden sein. Für einen Store-Miss legt nur Write-Allocate eine neue Line an. Der gezeigte C-Ausschnitt ist didaktisch und unvollständig. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 19 / 82

Array vs. Linked List

- Pointer-Knoten können verstreut liegen; das ist möglich, nicht zwingend
- Dann füllt eine Line Nachbardaten, die der nächste Knoten nicht nutzt
- Ein kompaktes Listenlayout kann räumliche Lokalität behalten

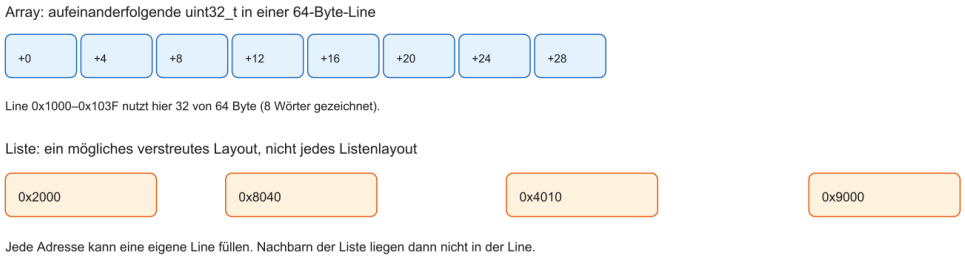


Abbildung 3: Array nutzt eine Line gemeinsam. Die Liste ist ein mögliches Streulayout.

Notizen

Array vs. Linked List schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Pointer-Knoten können verstreut liegen; das ist möglich, nicht zwingend. Dann füllt eine Line Nachbardaten, die der nächste Knoten nicht nutzt. Ein kompaktes Listenlayout kann räumliche Lokalität behalten. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 20 / 82

Layout im Speicher: Array vs. Liste

Die Abbildung steht auf der vorigen Folie: zusammenhängende Wörter gegen ein ausdrücklich mögliches Streulayout.

Notizen

Layout im Speicher: Array vs. Liste schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste

Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 21 / 82

Die 90/10-Regel

- Faustregel, workloadabhängig: oft liegt viel Laufzeit in wenig Code
- Der übrige Code wird nicht grundsätzlich nur einmal geladen

Notizen

Die 90/10-Regel schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Faustregel, workloadabhängig: oft liegt viel Laufzeit in wenig Code. Der übrige Code wird nicht grundsätzlich nur einmal geladen. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 22 / 82

Übung — Lokalität

Welche Folge ist zeitlich, welche räumlich: `0x1000, 0x1000` oder `0x1000, 0x1004`? Welche Loads stehen im gezeigten Assembler wirklich?

Notizen

Übung — Lokalität schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht

ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 23 / 82

Was ist ein Cache?

- Kleiner, schneller Speicher mit Kopien aus einem größeren, langsameren Speicher
- Transparent: Software sieht ihn in der Regel nicht (`load_to_cache` gibt es nicht)
- Steuerwerk kopiert, sucht und verwaltet autonom
- CPU fordert `0x1000` an — der Cache fängt die Anfrage ab

Notizen

Was ist ein Cache schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Kleiner, schneller Speicher mit Kopien aus einem größeren, langsameren Speicher. Transparent: Software sieht ihn in der Regel nicht (`load_to_cache` gibt es nicht). Steuerwerk kopiert, sucht und verwaltet autonom. CPU fordert `0x1000` an — der Cache fängt die Anfrage ab. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 24 / 82

Split Caches: L1i und L1d

- Von-Neumann-Flaschenhals auf L1-Ebene entschärfen
- Split Caches (Harvard auf L1):
  - L1-I: nur Befehle (Fetch)
  - L1-D: nur Daten (Memory)
- Vorteil: Fetch und Memory gleichzeitig, ohne Streit um denselben Port
- L2 oft unified (gemeinsam für Code und Daten)

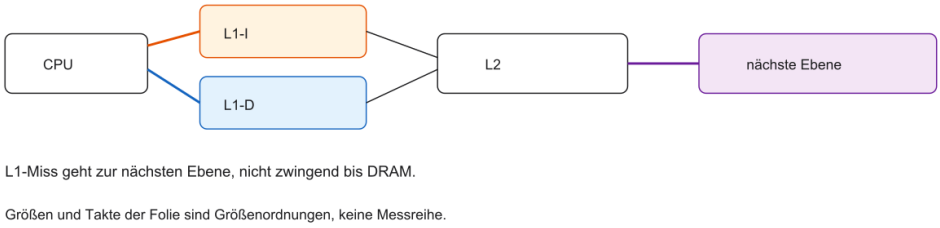


Abbildung 4: L1-I für Fetch, L1-D für Loads/Stores, gemeinsam zur nächsten Ebene.

Notizen

Split Caches: L1i und L1d schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Von-Neumann-Flaschenhals auf L1-Ebene entschärfen. Split Caches (Harvard auf L1): L1-I: nur Befehle (Fetch). L1-D: nur Daten (Memory). Vorteil: Fetch und Memory gleichzeitig, ohne Streit um denselben Port. L2 oft unified (gemeinsam für Code und Daten). Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 25 / 82

Cache Hit vs. Cache Miss

Cache Hit (Treffer):

- Adresse liegt als Kopie im Cache
- Lieferung in der Hit Time des getroffenen Levels
- Ein L1-Miss kann in L2 treffen; erst ein Miss der letzten Cache-Ebene geht zum DRAM

Notizen

Cache Hit vs. Cache Miss schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Adresse liegt als Kopie im Cache. Lieferung in der Hit Time des getroffenen Levels. Ein L1-Miss kann in L2 treffen; erst ein Miss der letzten Cache-Ebene geht zum DRAM. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 26 / 82

Hit Rate und Miss Rate

$$\text{Hit Rate} = \frac{\text{Hits}}{\text{Gesamtzugriffe}}, \qquad \text{Miss Rate} = 1 - \text{Hit Rate}$$

- Hit Rate: Anteil der Zugriffe, die im Cache gefunden werden
- Miss Rate: Anteil der Fehlschläge
- Praxiswerte hängen vom Workload ab; 90–98 % ist ein typischer Bereich, keine Garantie

Notizen

Hit Rate und Miss Rate schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Hit Rate: Anteil der Zugriffe, die im Cache gefunden werden. Miss Rate: Anteil der Fehlschläge. Praxiswerte hängen vom Workload ab; 90–98 % ist ein typischer Bereich, keine Garantie. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 27 / 82

Miss Penalty

- Hit Time: Kosten jedes Zugriffs im Modell, hier 1 Takt
- Miss Penalty: *zusätzliche* Kosten nur der Misses, hier 100 Takte
- Das 3-GHz-Beispiel „100 ns = 300 Takte“ ist eine andere Größenordnung, nicht diese Penalty

Notizen

Miss Penalty schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Hit Time: Kosten jedes Zugriffs im Modell, hier 1 Takt. Miss Penalty: *zusätzliche* Kosten nur der Misses, hier 100 Takte. Das 3-GHz-Beispiel „100 ns = 300 Takte“ ist eine andere Größenordnung, nicht diese Penalty. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 28 / 82

AMAT — Average Memory Access Time

Klausurrelevante Kernformel:

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

- Jeder Zugriff kostet mindestens die Hit Time (Cache wird immer geprüft)
- Nur bei Miss kommt die gewichtete Penalty hinzu

Notizen

AMAT heißt Average Memory Access Time, die mittlere Zugriffszeit der betrachteten Anfragen. Mit Hit Time t und zusätzlicher Miss Penalty p gilt AMAT = t + m·p, weil jeder Zugriff t bezahlt. Bei t = 1 Takt und p = 100 Takten kostet ein Miss 101 Takte. Das ist kein Worst Case und kein gemessener Programm-Speedup. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Rechenbeispiel: Macht des Caches

- L1 Hit Time: 1 Takt
- RAM Miss Penalty: 100 Takte

Ohne Cache in diesem Modell: jeder Zugriff 100 Takte.
Mit 95 Hits und 5 Misses:

$$95 \cdot 1 + 5 \cdot (1 + 100) = 600, \qquad 600/100 = 6$$



Abbildung 5: Alle 100 Zugriffe zahlen 1 Takt, fünf zahlen zusätzlich 100. AMAT = 6.

6 Takte sind die mittlere Zugriffszeit. $100/6$ ist ein Verhältnis dieser Zeiten, kein gemessener Speedup des Programms.

Notizen

95 Treffer mal 1 Takt plus 5 Fehlschläge mal 101 Takte ergeben 600 Takte. 600 geteilt durch 100 Zugriffe ist AMAT = 6 Takte. Die fünf Misses bezahlen die Prüfung ebenfalls. 100/6 vergleicht nur die modellierten Zugriffszeiten mit einem separaten Modell von 100 Takten ohne Cache. Die fünf Fehlschläge kosten je 101 Takte, zusammen 505. Die 95 Treffer kosten 95. Summe 600. Eine optionale zweite Ebene mit lokaler Miss Rate 0,20 und 10 Takten L2-Zeit ergäbe ein anderes Modell: 1 plus 0,05 mal (10 plus 0,20 mal 100) gleich 2,5 Takte. Die globale Miss Rate dahinter wäre 0,01. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Cache-Verwaltung in ganzen Lines

- Räumliche Lokalität ⇒ keine Einzelbyte-Verwaltung
- Verwaltungseinheit: Cache Block / Cache Line
- Typisch 32 oder 64 Bytes zusammenhängender Speicher
- Ein Miss beschafft die ganze, an der Blockgrenze ausgerichtete Line aus der nächsten zuständigen Ebene
- Ein L1-Miss kann in L2 treffen. DRAM ist nicht jede Quelle

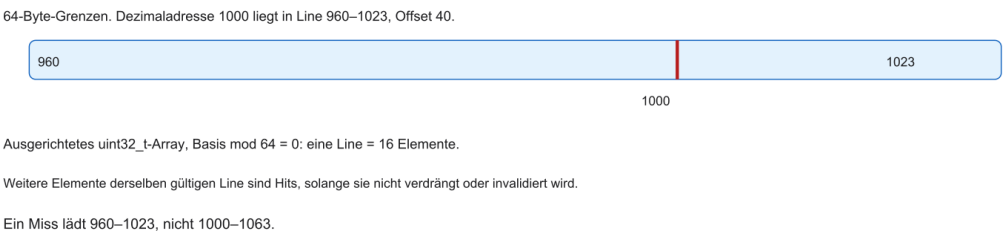


Abbildung 6: Adresse 1000 liegt in der Line 960–1023, Offset 40. Nicht 1000–1063.

Feld	Bedeutung
Valid	Eintrag wurde geladen
Tag	welcher Block
Daten	64 Byte auf Blockgrenze
Dirty	nur bei Write-Back: verändert

Notizen

Ein CPU-Load darf ein einzelnes Byte anfordern. Das Cache-Modell verwaltet und beschafft trotzdem eine ganze, an der Blockgrenze ausgerichtete Line. Nicht jede Region, insbesondere MMIO, ist cachebar. Dezimal 1000 liegt in der Line 960 bis 1023, Offset 40. Das ist nicht die Hexadresse 0x1000. floor(1000/64) ist 15, die Basis ist 960, das Ende ist 1023, der Offset ist 40. Ein Byte-Load der CPU ändert diese Verwaltungsgröße nicht. Liegt das erste Arrayelement nicht auf der Line-Grenze, passen nicht automatisch 16 Werte in dieselbe Line. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 31 / 82

Cache Line und räumliche Lokalität

- Miss bei `array[0]` lädt die ganze Line (z. B. 16 Integers)
- Weitere Elemente derselben gültigen Line sind Hits, solange die Line nicht verdrängt oder invalidiert wird
- „Garantierte Hits“ gibt es ohne diese Annahme nicht

Notizen

Cache Line und räumliche Lokalität schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Miss bei `array[0]` lädt die ganze Line (z. B. 16 Integers). Weitere Elemente derselben gültigen Line sind Hits, solange die Line nicht verdrängt oder invalidiert wird. „Garantierte Hits“ gibt es ohne diese Annahme nicht. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 32 / 82

Anatomie einer Cache-Zeile

Ein Cache-Eintrag enthält drei Felder:

1. Daten (Block): z. B. 64-Byte-Paket aus dem RAM
2. Valid-Bit: 1 = gültige Daten, 0 = leer/ungültig (nach Reset)
3. Tag: Adress-Etikett — woher aus dem großen Adressraum stammt der Block?

Valid	Tag	Daten
1 Bit	z. B. 17 Bit	z. B. 64 Bytes

Notizen

Anatomie einer Cache-Zeile schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Valid Tag Daten. 1 Bit z. B. 17 Bit z. B. 64 Bytes. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain,

Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 33 / 82

Blockgröße: Der Kompromiss

- Größere Lines: bessere räumliche Lokalität — aber höhere Miss Penalty
- Wenige große Blöcke: weniger „Slots“, mehr Verdrängung (Thrashing)
- Industrie-Sweetspot: oft 32–64 Bytes pro Line

Notizen

Blockgröße: Der Kompromiss schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Größere Lines: bessere räumliche Lokalität — aber höhere Miss Penalty. Wenige große Blöcke: weniger „Slots“, mehr Verdrängung (Thrashing). Industrie-Sweetspot: oft 32–64 Bytes pro Line. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 34 / 82

Übung — AMAT

Berechnen Sie AMAT für Hit Rate 95 % und 80 % bei Hit Time 1 und zusätzlicher Penalty 100. Was bedeutet „zusätzlich“?

Notizen

Bei 95 Prozent Hit Rate ist die Miss Rate 0,05 und AMAT = 1 + 0,05·100 = 6 Takte. Bei 80 Prozent ist die Miss Rate 0,20 und AMAT = 21 Takte. AMAT ist nicht 5 oder 20, weil die Hit Time bei jedem Zugriff anfällt. Beide Fälle nutzen dieselbe Hit Time und dieselbe zusätzliche Penalty. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 35 / 82

Das Mapping-Problem

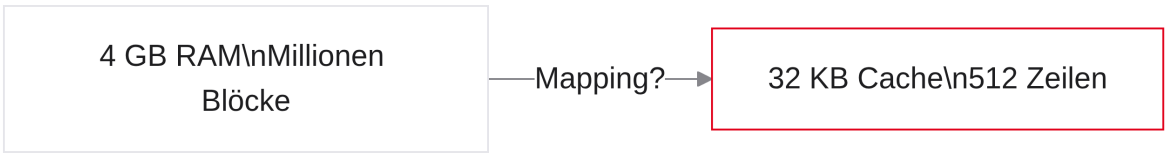
- Massives Platzproblem zwischen RAM und Cache
- RAM: z. B. 4 GB (Millionen Cache-Blöcke)
- L1-Cache: z. B. 32 KB (vielleicht nur 512 Cache-Zeilen)
- Frage: Wo darf ein RAM-Block im Cache abgelegt werden?
- Herausforderung: Mapping in Hardware, idealerweise in höchstens 1 Takt
- Drei grundlegende Strategien lösen dieses Problem

Notizen

Das Mapping-Problem schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Massives Platzproblem zwischen RAM und Cache. RAM: z. B. 4 GB (Millionen Cache-Blöcke). L1-Cache: z. B. 32 KB (vielleicht nur 512 Cache-Zeilen). Frage: Wo darf ein RAM-Block im Cache abgelegt werden? Herausforderung: Mapping in Hardware, idealerweise in höchstens 1 Takt. Drei grundlegende Strategien lösen dieses Problem. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 36 / 82

Mapping: Kapazität vs. Suche



Notizen

Mapping: Kapazität vs. Suche schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode.

Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 37 / 82

Direct Mapped Cache

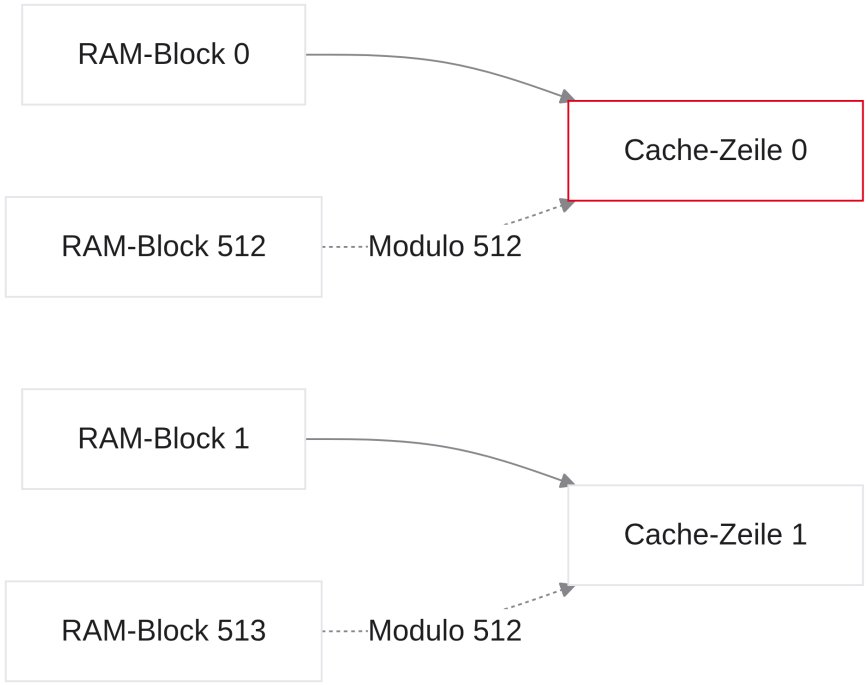
- Einfachste Strategie: Direct Mapped (Direktabbildung)
- Regel: jeder RAM-Block hat genau einen festen Platz im Cache
- Parkhaus-Metapher: Parkplatznummer aus den letzten Ziffern des Nummernschilds
- Endet das Schild auf 05 → nur Parkplatz 05
- Vorteil: Suche nur an genau diesem einen Platz

Notizen

Direct Mapped Cache schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Einfachste Strategie: Direct Mapped (Direktabbildung). Regel: jeder RAM-Block hat genau einen festen Platz im Cache. Parkhaus-Metapher: Parkplatznummer aus den letzten Ziffern des Nummernschilds. Endet das Schild auf 05 → nur Parkplatz 05. Vorteil: Suche nur an genau diesem einen Platz. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 38 / 82

Direct Mapped: feste Zuordnung



Notizen

Direct Mapped: feste Zuordnung schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 39 / 82

Modulo-Arithmetik in Hardware

- Cache-Index $=$ (RAM-Blockadresse) mod (Anzahl Cache-Zeilen)
- Modulo einer Zweierpotenz ist ein Bitfeld, kein allgemeiner Dividierer
- Die Auswahl selbst ist verdrahtet; der Speicherzugriff ist deshalb nicht verzögerungsfrei

Notizen

Modulo-Arithmetik in Hardware schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Cache-Index $=$ (RAM-Blockadresse) mod (Anzahl Cache-Zeilen). Modulo einer Zweierpotenz ist ein Bitfeld, kein allgemeiner Dividierer. Die Auswahl selbst ist verdrahtet; der Speicherzugriff ist deshalb nicht verzögerungsfrei. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 40 / 82

Die Adress-Zerlegung

Die CPU legt eine 32-Bit-Adresse auf den Bus. Der Cache zerschneidet sie in drei Felder:

1. Block Offset — unterste Bits
2. Cache Index — mittlere Bits
3. Tag — oberste Bits

Diese drei Felder sind der Schlüssel jeder Cache-Architektur.

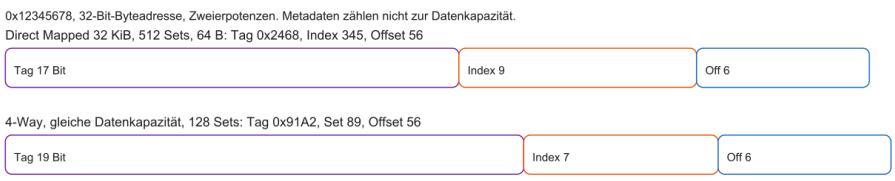


Abbildung 7: Referenzmodell: 17/9/6 bei Direct Mapped, 19/7/6 bei 4-Way, Adresse 0x12345678.

- $S = C/(B \cdot W)$, Offset $\log_2 B$, Index $\log_2 S$, Tag $P - b - s$
- Zweierpotenzen erlauben die Zerlegung durch Bitfelder, ohne Dividierer
- Leitungen, Decoder, SRAM, Komparator und MUX haben trotzdem Laufzeit

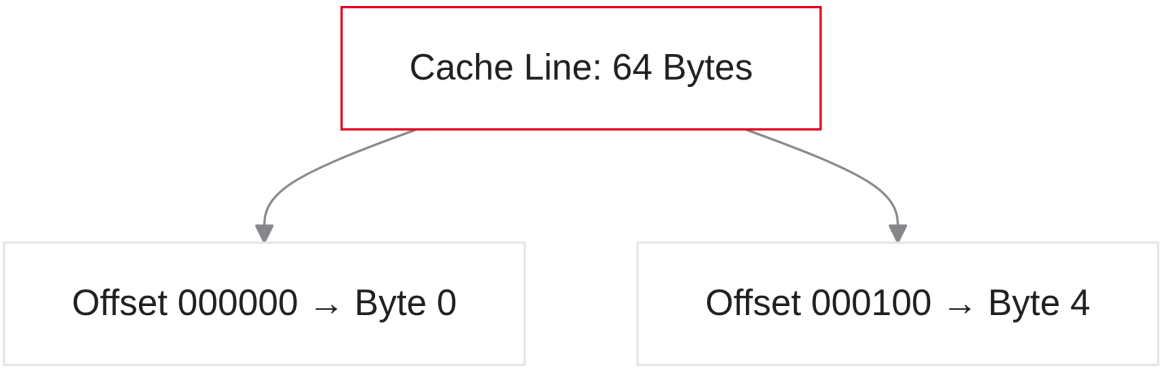
Notizen

Die Adress-Zerlegung schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. $S = C/(B \cdot W)$, Offset $\log_2 B$, Index $\log_2 S$, Tag $P - b - s$. Zweierpotenzen erlauben die Zerlegung durch Bitfelder, ohne Dividierer. Leitungen, Decoder, SRAM, Komparator und MUX haben trotzdem Laufzeit. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 41 / 82

Feld 1: Der Block Offset

- Wo in der Cache-Zeile liegt das gesuchte Byte?
- Eine Line bringt z. B. 64 Bytes am Stück
- CPU will oft nur ein 4-Byte-Wort
- $2^6 = 64 \rightarrow$ unterste 6 Bits als Offset innerhalb des Blocks



Notizen

Feld 1: Der Block Offset schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Wo in der Cache-Zeile liegt das gesuchte Byte? Eine Line bringt z. B. 64 Bytes am Stück. CPU will oft nur ein 4-Byte-Wort. $2^6 = 64 \rightarrow$ unterste 6 Bits als Offset innerhalb des Blocks. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 42 / 82

Feld 2: Der Cache Index

- Welche Zeile des Caches? (Modulo-Operation)
- Mittlere Bits schneiden die Adresse auf die Cache-Größe
- Beispiel: 512 Zeilen $\Rightarrow 2^9 \rightarrow$ nächste 9 Bits über dem Offset
- Diese Bits gehen direkt an den Adress-Decoder des Cache-SRAM

Notizen

Bei 512 Sets genügen neun Indexbits, bei 128 Sets im Vier-Wege-Vergleich sieben. Der Decoder wählt das Set, nicht den Treffer. Decoder, SRAM, Vergleich und Multiplexer brauchen reale Zeit. Für 0x12345678 wählt das Direct-Mapped-Modell Index 345 von 512. Der Decoder liest diese eine Zeile. Erst Tag und Valid entscheiden den Treffer. Bei gleicher Datenkapazität und vier Wegen bleiben 128 Sets, also sieben Indexbits und Set 89. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 43 / 82

Feld 3: Das Tag

- Prüft: Sind das wirklich meine Daten?
- Alias-Problem: Millionen RAM-Blöcke teilen denselben Index
- Beim Öffnen der Zeile: welches der vielen Blöcke liegt darin?
- Oberste Bits = Tag
- Bei 512 Sets hat Block 512 den Tag 1, nicht das Etikett 512
- Der Vergleich ist ein Bitvergleich mit Reduktion, nicht ein einzelnes XOR als vollständige Schaltung

Notizen

Feld 3: Das Tag schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Prüft: Sind das wirklich meine Daten? Alias-Problem: Millionen RAM-Blöcke teilen denselben Index. Beim Öffnen der Zeile: welches der vielen Blöcke liegt darin? Oberste Bits = Tag. Bei 512 Sets hat Block 512 den Tag 1, nicht das Etikett 512. Der Vergleich ist ein Bitvergleich mit Reduktion, nicht ein einzelnes XOR als vollständige Schaltung. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis.

Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 44 / 82

Das Valid-Bit — Boot-Problem

- Nach Einschalten: SRAM voller zufälligem Rauschen
- Zufällig könnte das Rauschen dem gesuchten Tag gleichen
- Lösung: 1-Bit Valid-Flag
- Hardware-Reset setzt alle Valid-Bits auf 0
- Hit nur wenn Tag übereinstimmt und Valid $= 1$

Notizen

Das Valid-Bit — Boot-Problem schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Nach Einschalten: SRAM voller zufälligem Rauschen. Zufällig könnte das Rauschen dem gesuchten Tag gleichen. Lösung: 1-Bit Valid-Flag. Hardware-Reset setzt alle Valid-Bits auf 0. Hit nur wenn Tag übereinstimmt und Valid $= 1$. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 45 / 82

Ablauf eines Lesezugriffs (Direct Mapped)

1. 32-Bit-Adresse anliegend — Index-Bits öffnen das Cache-SRAM
2. Gespeichertes Tag und Valid-Bit auslesen
3. Komparator: $(\text{Tag} = \text{CPU-Tag}) \wedge (\text{Valid} = 1)$
4. Hit → Daten zur CPU; Miss → Stall und Anfrage an Main Memory

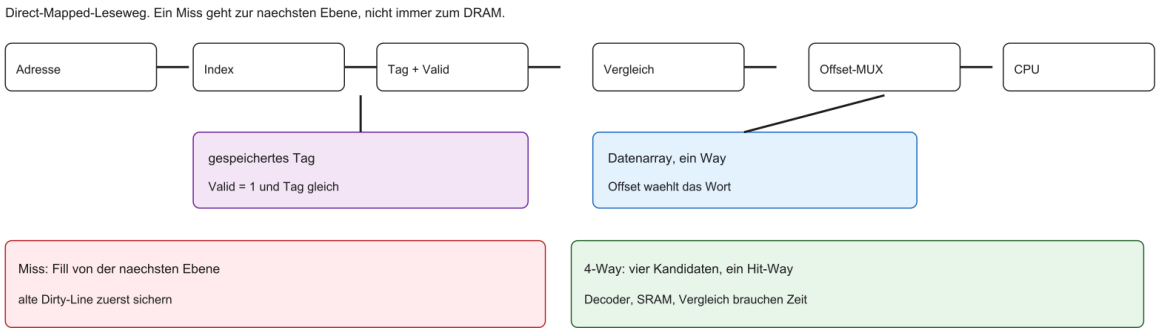


Abbildung 8: Index öffnet das Set, Tag und Valid vergleichen, Offset wählt das Wort, Daten gehen zur CPU.

Notizen

Zuerst wird die Adresse zerlegt, dann das eine Set gelesen, Tag und Valid geprüft. Ein Treffer liefert den Offset-Bereich. Ein Fehlschlag fragt die nächste zuständige Ebene, nicht grundsätzlich DRAM. Der Vier-Wege-Vergleich ist eine spätere Erweiterung, nicht der titelgebende Ablauf. Die Adresse zerfällt in Tag, Index und Offset. Der Index liest eine Zeile. Hit bedeutet gültiges passendes Tag, dann liefert der Offset das angeforderte Wort. Beim Miss füllt die nächste Ebene diese eine Zeile. Die abhängige Pipeline wartet im blockierenden Modell. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 46 / 82

Vorteil von Direct Mapped

- Suche nur an einer Stelle → minimaler Hardware-Aufwand
- Nur ein Komparator: Strom und Chipfläche sparen
- Schnelle Hit-Time: oft Antwort in einem Takt
- Bei Miss keine Wahl: nur ein Platz kann verdrängt werden

Notizen

Vorteil von Direct Mapped schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Suche nur an einer Stelle → minimaler Hardware-Aufwand. Nur ein Komparator: Strom und Chipfläche sparen. Schnelle Hit-Time: oft Antwort in einem Takt. Bei Miss keine Wahl: nur ein Platz kann verdrängt werden. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Nachteil: Thrashing

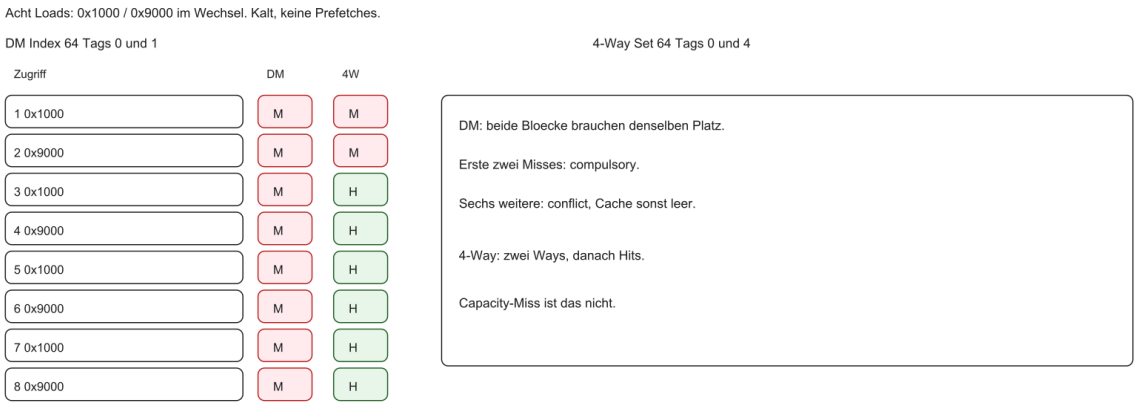


Abbildung 9: Acht Zugriffe auf 0x1000 und 0x9000. Direct Mapped: 8 Misses. 4-Way: 2 Misses und 6 Hits.

- Beide Adressen treffen Set 64. Direct-Mapped-Tags 0 und 1, 4-Way-Tags 0 und 4
- Zwei compulsory Misses, im Direct Mapped zusätzlich sechs Conflict Misses
- Der übrige Cache kann leer bleiben; das ist kein Capacity Miss dieses Paars

Notizen

Nachteil: Thrashing schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Beide Adressen treffen Set 64. Direct-Mapped-Tags 0 und 1, 4-Way-Tags 0 und 4. Zwei compulsory Misses, im Direct Mapped zusätzlich sechs Conflict Misses. Der übrige Cache kann leer bleiben; das ist kein Capacity Miss dieses Paars. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 48 / 82

Thrashing-Beispiel in C

Thrashing *kann* auftreten, wenn zwei heiß genutzte Arrays denselben Cache-Index treffen (abhängig von Alignment, Array-Größe und Cache-Organisation):

```
// Kollision möglich bei gleichem Cache-Index
int A[1000];
int B[1000];

for (int i = 0; i < 1000; i++) {
    // Konflikt: gegenseitiges Verdrängen
    A[i] = A[i] + B[i];
}
```

Notizen

Thrashing-Beispiel in C schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 49 / 82

Übung — Adressfelder

Bestimmen Sie für 0x12345678 im Referenzmodell Tag, Index und Offset, direct mapped. Was liefert die Folge 0x1000, 0x9000, 0x1000 im kalten Direct-Mapped-Cache?

Notizen

Für 0x12345678 im Direct-Mapped-Modell gilt Offset 56, Index 345 und Tag 0x2468. Die Folge 0x1000, 0x9000, 0x1000 hat dreimal Index 64 und die Tags 0, 1, 0. Im kalten Cache sind das drei Misses, nicht zweimal Set 64. Rechnung: Blocknummer ist floor(Adresse/64). Index ist diese Nummer modulo 512, das Tag der ganzzahlige Quotient. Offset ist der Rest der Adresse modulo 64. Für 0x1000 und 0x9000 ist der Abstand 32768 Byte, also genau eine Direct-Mapped-Kapazität. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und

der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 50 / 82

Fully Associative Cache

- Gegenteil von Direct Mapped: Fully Associative (vollassoziativ)
- Regel: RAM-Block darf an jedem freien Platz liegen
- Parkhaus: freie Platzwahl
- Vorteil: die Indexkollision von A und B entfällt. Capacity Misses bleiben möglich

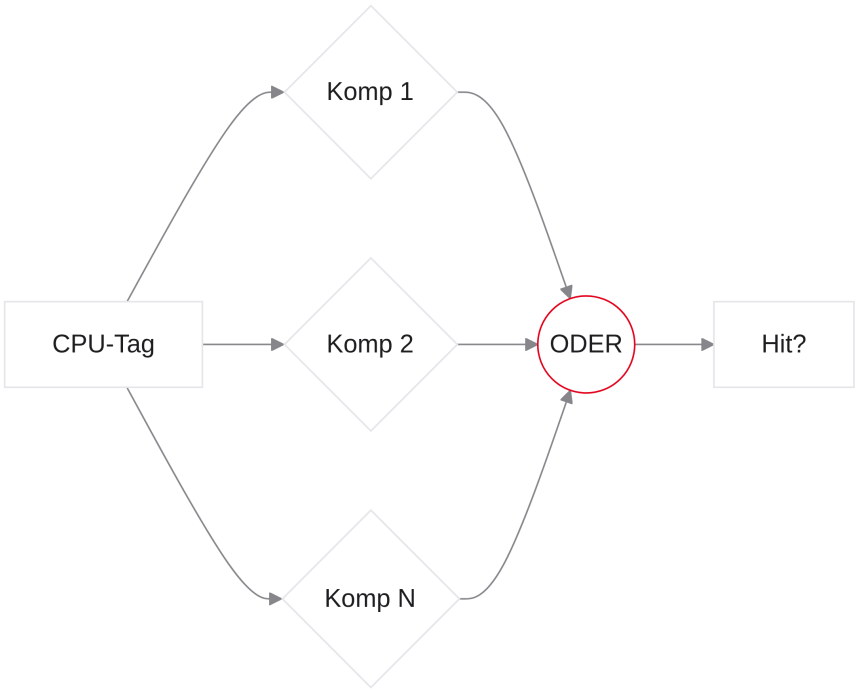
Notizen

Jeder Block darf in jedem Platz liegen. Dadurch entfällt der Indexkonflikt von 0x1000 und 0x9000. Ein zu großes Working Set kann weiterhin verdrängen. Compulsory Misses bleiben. Vollassoziativität maximiert nicht allgemein die Hit Rate und ist nicht unbrauchbar. Im Referenzmodell gibt es 512 Datenplätze und ein einziges Set. Die acht Zugriffe auf 0x1000 und 0x9000 brauchen dann keine festen Indexplätze. Ein Working Set aus 513 verschiedenen Lines passt trotzdem nicht gleichzeitig hinein. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 51 / 82

Das Such-Problem bei Fully Associative

- Freiheit hat einen Preis: Wo liegt der Block?
- Kein Index mehr — Adresse nur noch Tag und Offset
- Hardware muss alle Zeilen parallel nach dem Tag durchsuchen
- Bei 512 Zeilen: 512 parallele Komparatoren



Notizen

Das Such-Problem bei Fully Associative schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Freiheit hat einen Preis: Wo liegt der Block? Kein Index mehr — Adresse nur noch Tag und Offset. Hardware muss alle Zeilen parallel nach dem Tag durchsuchen. Bei 512 Zeilen: 512 parallele Komparatoren. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 52 / 82

Nachteile von Fully Associative

- Strom und Hitze: hunderte parallele Vergleiche jede Nanosekunde
- Chipfläche: Komparatoren oft größer als der SRAM selbst
- Latenz: große ODER-Netze verlangsamen das Hit-Signal
- Fazit: für große L1-Caches in der Praxis unbrauchbar

Notizen

Nachteile von Fully Associative schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Strom und Hitze: hunderte parallele Vergleiche jede Nanosekunde. Chipfläche: Komparatoren oft größer als der SRAM selbst. Latenz: große ODER-Netze verlangsamen das Hit-Signal. Fazit: für große L1-Caches in der Praxis unbrauchbar. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 53 / 82

Der Mittelweg: Set-Associative Cache

- Industriestandard: Set-Associative (mengenassoziativ)
- Hybrid aus Direct Mapped und Fully Associative
- Parkhaus: feste Zone (Set) wie Direct Mapped — darin z. B. 4 freie Plätze (Ways)
- Sie müssen in Zone 05 parken, dürfen dort aber einen der 4 Plätze wählen

Notizen

Der Mittelweg: Set-Associative Cache schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Industriestandard: Set-Associative (mengenassoziativ). Hybrid aus Direct Mapped und Fully Associative. Parkhaus: feste Zone (Set) wie Direct Mapped — darin z. B. 4 freie Plätze (Ways). Sie müssen in Zone 05 parken, dürfen dort aber einen der 4 Plätze wählen. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur

als Wiederholung lesen.

Folie 54 / 82

N-Way Set-Associative Struktur

- Cache in Sets; jedes Set hat N Ways
- Beispiel 4-Way: Index zeigt auf Set mit 4 Blöcken
- Suche: nur N parallele Komparatoren (z. B. 4)
- Bestes aus beiden Welten:
  - wenig Komparatoren (schnell, sparsam)
  - bis zu N Blöcke mit gleichem Index ohne gegenseitiges Rauswerfen

Index	Way 0	Way 1	Way 2	Way 3
Set 0	Block	Block	Block	Block
Set 1	…	…	…	…

Notizen

N-Way Set-Associative Struktur schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Cache in Sets; jedes Set hat N Ways. Beispiel 4-Way: Index zeigt auf Set mit 4 Blöcken. Suche: nur N parallele Komparatoren (z. B. 4). Bestes aus beiden Welten: wenig Komparatoren (schnell, sparsam). bis zu N Blöcke mit gleichem Index ohne gegenseitiges Rauswerfen. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 55 / 82

Praxis: 2-Way und 4-Way

- Moderne CPUs nutzen typischerweise Set-Associative Caches
- Typische Größenordnungen (schematisch, je nach Generation/Produkt):
  - L1: oft 4-Way oder 8-Way
  - L3: oft höhere Assoziativität (z. B. 12–16-Way)
- Größeres N: weniger Konflikt-Misses, aber mehr Komparatoren und Strom

Angaben schematisch — konkrete Werte den jeweiligen Microarchitecture-/Datenblättern entnehmen.

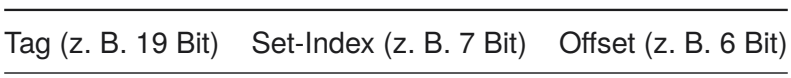
Notizen

Praxis: 2-Way und 4-Way schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Moderne CPUs nutzen typischerweise Set-Associative Caches. Typische Größenordnungen (schematisch, je nach Generation/Produkt): L1: oft 4-Way oder 8-Way. L3: oft höhere Assoziativität (z. B. 12–16-Way). Größeres N: weniger Konflikt-Misses, aber mehr Komparatoren und Strom. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 56 / 82

Adress-Zerlegung bei Set-Associative

- Ein Set fasst mehrere Ways → insgesamt weniger Sets
- Weniger Sets → weniger Index-Bits
- Eingesparte Index-Bits wandern ins Tag
- Offset bleibt gleich (Blockgröße unverändert)



Notizen

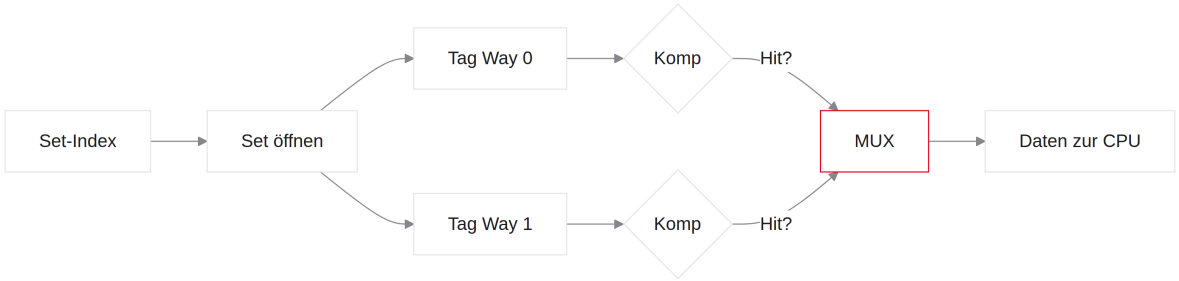
Adress-Zerlegung bei Set-Associative schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ein Set fasst mehrere Ways → insgesamt weniger Sets. Weniger Sets → weniger Index-Bits. Eingesparte Index-Bits wandern ins Tag. Offset bleibt gleich (Blockgröße unverändert). Tag (z. B. 19 Bit) Set-Index (z. B. 7 Bit) Offset (z. B. 6 Bit). Jede Zahl

gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 57 / 82

Hardware-Ablauf beim N-Way Cache

1. Set-Index wählt genau ein Set
2. Alle N Ways (Tags + Valid) parallel auslesen
3. N Komparatoren vergleichen mit dem CPU-Tag
4. MUX wählt die Datenleitung des Hit-Ways



Notizen

Hardware-Ablauf beim N-Way Cache schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 58 / 82

Eviction — Verdrängung

- Direct Mapped: voller Platz → alter Block fliegt raus (keine Wahl)
- N-Way: Set voll (alle N Ways belegt) → welcher Block wird geopfert?
- Eviction = Verdrängung
- Hardware braucht eine Replacement Policy

Notizen

Eviction — Verdrängung schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Direct Mapped: voller Platz → alter Block fliegt raus (keine Wahl). N-Way: Set voll (alle N Ways belegt) → welcher Block wird geopfert? Eviction = Verdrängung. Hardware braucht eine Replacement Policy. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 59 / 82

Replacement: Random und FIFO

Random:

- Zufälligen Way aus dem Set werfen
- Hardware sehr einfach; bei großen Caches oft erstaunlich gut

FIFO (First-In-First-Out):

- Ältesten geladenen Block verdrängen
- Problem: früh geladene, weiter genutzte Blöcke können trotzdem das Victim sein

Notizen

FIFO merkt sich die Ladereihenfolge, nicht die letzte Nutzung. Ein Schleifenzähler ist kein belastbares Beispiel, weil er in einem Register bleiben kann. Die Spur A, B, C, D, A, E zeigt den Unterschied zu LRU an echten Blocknamen. Nach A, B, C, D, A bleibt die FIFO-Reihenfolge A, B, C, D. E verdrängt A. LRU hat nach dem Hit auf A die Reihenfolge B, C, D, A und verdrängt B. Random braucht einen Auswahlmechanismus und ist nicht kostenfrei. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er

nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 60 / 82

Replacement: LRU

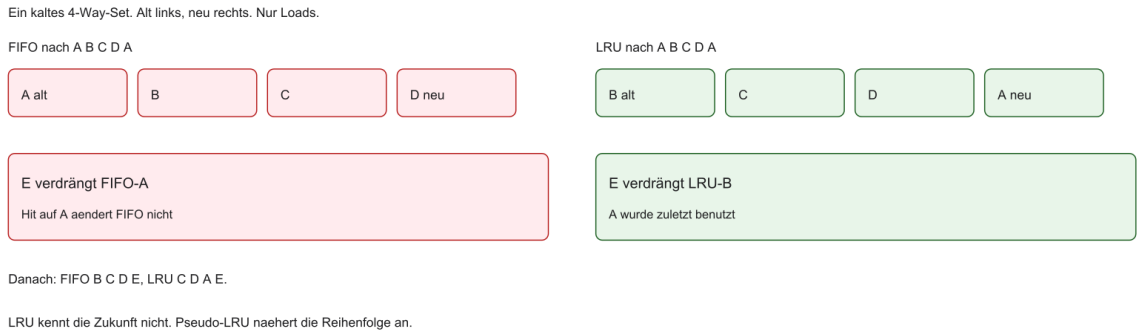


Abbildung 10: Folge A B C D A E in einem 4-Way-Set. FIFO verdrängt A, LRU verdrängt B.

- Ein Hit macht den Block bei LRU wieder jung, bei FIFO bleibt die Ladereihenfolge
- LRU ist eine gute Heuristik, nicht perfekt und nicht immer die beste Policy
- Zeitstempel sind eine mögliche Umsetzung. Pseudo-LRU spart Bits

Notizen

Replacement: LRU schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ein Hit macht den Block bei LRU wieder jung, bei FIFO bleibt die Ladereihenfolge. LRU ist eine gute Heuristik, nicht perfekt und nicht immer die beste Policy. Zeitstempel sind eine mögliche Umsetzung. Pseudo-LRU spart Bits. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 61 / 82

Zusammenfassung Mapping-Strategien

Merkmal	Direct Mapped	Fully Associative	N-Way Set-Associative
Platzwahl	1 fester Platz	überall	frei innerhalb eines Sets
Komparatoren	1	sehr viele	N (z. B. 4)
Kollisionen	Conflict Misses möglich	keine Conflict Misses	weniger pro Set
Eviction	keine Wahl	LRU / Random	LRU / Random
Praxis	billige Chips	kleine Puffer (z. B. TLB)	L1/L2/L3-Standard

Notizen

Zusammenfassung Mapping-Strategien schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Merkmal Direct Mapped Fully Associative N-Way Set-Associative. Platzwahl 1 fester Platz überall frei innerhalb eines Sets. Komparatoren 1 sehr viele N (z. B. 4). Kollisionen Conflict Misses möglich keine Conflict Misses weniger pro Set. Eviction keine Wahl LRU / Random LRU / Random. Praxis billige Chips kleine Puffer (z. B. TLB) L1/L2/L3-Standard. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 62 / 82

Übung — gleiche Kapazität

32 KiB und 64-Byte-Lines: Warum hat 4-Way weniger Indexbits als Direct Mapped?

Notizen

512 Lines zu je 64 Byte sind 32 KiB, sowohl als 512 mal 1 als auch als 128 mal 4. Bei 32-Bit-Adressen sind die Felder 17/9/6 beziehungsweise 19/7/6 Bit. Metadaten und Replacement-Zustand zählen nicht zur Datenkapazität. KiB bedeutet 1024 Byte. 32 KiB geteilt durch 64 Byte sind 512 Lines. Vier Wege teilen diese Lines auf 128 Sets. log2(64) ist 6, log2(512) ist 9, log2(128) ist 7. Die Tagbreite wächst von 17 auf 19 Bit, die Datenkapazität bleibt 32 KiB. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode.

Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 63 / 82

Das Problem beim Schreiben

- Bisher: Lesen (`lw`)
- Store (`sw`): CPU schreibt einen neuen Wert
- Schreiben in den L1 ist schnell — der DRAM bleibt aber unverändert
- Cache hält dann einen neueren Wert als der RAM

Notizen

Nach einem Write-Back-Store kann der Cache 5 und die untere Ebene 0 enthalten. Das ist der reguläre Zustand dieser Strategie, kein Kohärenzfehler an sich. Der schreibende Kern muss seine eigenen folgenden Loads korrekt sehen. Andere Agenten brauchen eigene Sichtbarkeitsregeln. sw schreibt ein Wort. Bei Write-Back bleibt die untere Ebene zunächst alt. Der eigene Kern sieht den neuen Wert aus seiner Line. Ein zweiter Kern oder nicht kohärentes DMA sieht ihn nur nach den Regeln seines Protokolls. Dirty ist Verwaltungszustand, kein Fehler. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 64 / 82

Das Cache-Kohärenz-Problem

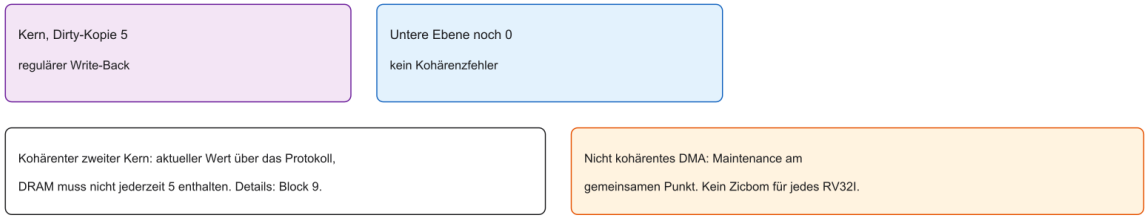


Abbildung 11: Dirty-Kopie 5 bei altem Wert 0 ist zunächst Write-Back, kein Kohärenzfehler. Kohärente Leser und nicht kohärentes DMA sind getrennte Fälle.

- Ein kohärenter zweiter Kern kann den aktuellen Wert über das Protokoll sehen
- Nicht kohärentes DMA braucht Sichtbarkeit am gemeinsamen Zugriffspunkt
- Clean, Invalidate und Flush sind konzeptionell verschieden und plattformabhängig
- Kein Zicbom und kein MESI für eine beliebige RV32I-Plattform; Multicore folgt in Block 9

Notizen

Das Cache-Kohärenz-Problem schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ein kohärenter zweiter Kern kann den aktuellen Wert über das Protokoll sehen. Nicht kohärentes DMA braucht Sichtbarkeit am gemeinsamen Zugriffspunkt. Clean, Invalidate und Flush sind konzeptionell verschieden und plattformabhängig. Kein Zicbom und kein MESI für eine beliebige RV32I-Plattform; Multicore folgt in Block 9. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 65 / 82

Schreibstrategie 1: Write-Through

- Write-Through (Durchschreiben)
- Jeder Schreibvorgang wird an die nächste Ebene weitergegeben
- Ohne Puffer wartet der Store auf diese Weitergabe. Mit Puffer kann unten noch der alte Wert stehen
- Ein anderer Kern kann trotzdem eine eigene Kopie halten

Notizen

Write-Through gibt den Store an die nächste Ebene weiter. Ohne Puffer ist der Store erst abgeschlossen, wenn diese Weitergabe im Modell fertig ist. Mit Puffer kann die CPU weiterlaufen, während unten noch der alte Wert steht. Write-Through invalidiert nicht automatisch die Kopie eines anderen Kerns. Store 5 und danach Store 7 starten bei Cachewort 0 und unterer Ebene 0. Ohne Puffer steht unten nach dem ersten Abschluss 5 und nach dem zweiten 7. Mit Puffer kann die CPU beide Stores annehmen, während unten noch 0 steht. Nach dem Abfluss ohne Write-Merging steht unten 7. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 66 / 82

Write-Through: Datenfluss

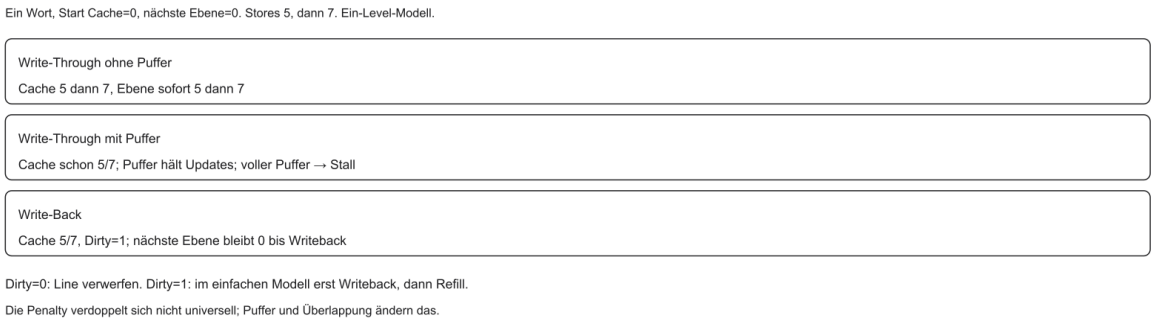


Abbildung 12: Stores 5 und 7: sofort, gepuffert oder nur im Cache mit Dirty.

Notizen

Write-Through: Datenfluss schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort.

Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 67 / 82

Nachteile von Write-Through

- Ohne Write Buffer: jedes `sw` wartet auf den langsamen DRAM (≈ 300 Takte)
- Rückfall in die Memory Wall beim Schreiben
- Schreibintensive Programme stark ausgebremst
- Bus durch ständige Einzel-Updates überlastet

Notizen

Nachteile von Write-Through schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ohne Write Buffer: jedes `sw` wartet auf den langsamen DRAM (≈ 300 Takte). Rückfall in die Memory Wall beim Schreiben. Schreibintensive Programme stark ausgebremst. Bus durch ständige Einzel-Updates überlastet. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 68 / 82

Hardware-Hack: Write Buffer

- Write Buffer: kleine FIFO-Warteschlange zwischen L1 und RAM
- CPU schreibt rasch in Cache und Buffer und rechnet weiter
- Buffer schickt Daten im Hintergrund zum DRAM
- Wenn der Buffer voll läuft: CPU muss wieder stallen

Notizen

Hardware-Hack: Write Buffer schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Write Buffer: kleine FIFO-Warteschlange zwischen L1

und RAM. CPU schreibt rasch in Cache und Buffer und rechnet weiter. Buffer schickt Daten im Hintergrund zum DRAM. Wenn der Buffer voll läuft: CPU muss wieder stallen. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

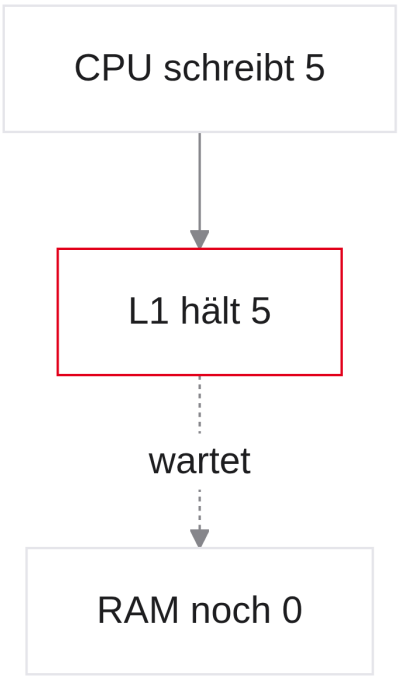
Schreibstrategie 2: Write-Back

- Write-Back (Zurückschreiben): Schreiben nur in den L1
- CPU kann dieselbe Variable tausendfach schnell aktualisieren
- Risiko: Cache ist „schmutzig“ — neuer als der RAM

Notizen

Ein Store-Hit aktualisiert die vorhandene Line und setzt Dirty. Die nächste Ebene bleibt zunächst unverändert. Das spart Verkehr bei wiederholten Stores derselben Line. Ob ein Store-Miss die Line überhaupt anlegt, entscheidet die Allocation-Policy erst danach. Starten Sie mit einer gültigen sauberen Line, Cachewort 0 und untere Ebene 0. Store 5 setzt das Cachewort auf 5 und Dirty auf 1. Store 7 setzt das Cachewort auf 7. Unten steht weiter 0, bis ein Writeback die aktuelle Line überträgt. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Write-Back: Datenfluss



Notizen

Write-Back: Datenfluss schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 71 / 82

Das Dirty-Bit

- Dirty-Bit (D) markiert veränderte Cache-Zeilen
- Nach Laden aus dem RAM: Dirty $= 0$ (sauber)
- Nach erstem $\mathtt{sw}$ in den Block: Dirty $= 1$

Valid	Dirty	Tag	Daten
1 Bit	1 Bit	Adress-Etikett	z. B. 64 Bytes

Notizen

Das Dirty-Bit schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Dirty-Bit (D) markiert veränderte Cache-Zeilen. Nach Laden aus dem RAM: Dirty $= 0$ (sauber). Nach erstem $\mathtt{sw}$ in den Block: Dirty $= 1$. Valid Dirty Tag Daten. 1 Bit 1 Bit Adress-Etikett z. B. 64 Bytes. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 72 / 82

Eviction bei Write-Back

- Writebacks entstehen durch Eviction, Maintenance oder Protokollaktionen, nicht nur durch Eviction
- Dirty = 0: die Line ist gegenüber der unteren Ebene sauber und darf verworfen werden
- Dirty = 1: der aktuelle Inhalt muss vor dem Verwerfen gesichert werden
- Puffer und Überlappung können die Penalty ändern; sie verdoppelt sich nicht allgemein

Notizen

Eine saubere Line darf verworfen werden. Eine veränderte Line muss vorher an ihre alte Adresse zurück. Die Rückschreibadresse entsteht aus altem Tag, Set-Index und Offset null, nicht aus der neuen Anfrage. Im Beispiel ersetzt 0x9000 die Dirty-Line 0x1000. Der Writeback geht nach 0x1000. Rekonstruktion: ((alter Tag mal 512) plus Index) mal 64. Mit Tag 0 und Index 64 ergibt das 0x1000. Die neue Anfrage 0x9000 ist die Fill-Adresse danach. Im seriellen Modell kommt der Writeback vor dem neuen Fill. Die Penalty verdoppelt sich dadurch nicht automatisch. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten

Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 73 / 82

Vergleich Schreibstrategien

Merkmal	Write-Through	Write-Back
Weitergabe	synchron oder über Puffer	bis Writeback
Store-Miss	eigene Policy	eigene Policy
Bus-Last im einfachen Modell	höher	geringer
Zusatzzustand	gering	Dirty-Bit

Write-Through und Write-Back bestimmen die Weitergabe geänderter Cache-Daten, nicht die RAM-Kohärenz.

Notizen

Write-Through und Write-Back sind die Weitergabe. Write-Allocate und No-Write-Allocate sind eine zweite Achse. Ein Puffer kann den unteren Speicher hinterherhinken lassen. Write-Back braucht das Dirty-Bit. Keine der beiden Strategien stellt allein Mehrkern- oder DMA-Kohärenz her. Mehrkern- und DMA-Sichtbarkeit folgen eigenen Regeln. Ungepuffertes Write-Through schließt den Store erst nach der Weitergabe ab. Der Puffer kann oben schon 7 halten, während unten noch 0 steht. Write-Back bündelt mehrere Stores derselben Line. Die Store-Miss-Policy entscheidet unabhängig, ob die fehlende Line geladen wird. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 74 / 82

Store-Miss: Write-Allocate



Abbildung 13: Weitergabe der Writes und Behandlung eines Store-Miss sind zwei Achsen.

- Write-Allocate: fehlenden Block laden, dann den Store im Cache ausführen
- No-Write-Allocate: Store weitergeben und keinen neuen Cache-Eintrag anlegen
- Typische Paarungen sind Beispiele, keine zwingende Kopplung

Notizen

Allocate lädt die fehlende Line und führt den Store dort aus. No-Allocate gibt den Store weiter und legt keinen neuen Eintrag an. Dirty gleich 0 heißt sauber und darf verworfen werden. Dirty gleich 1 verlangt, den aktuellen Inhalt vor dem Verwerfen zu sichern. Ein einzelnes sichtbares Wort ersetzt nicht den gesicherten ganzen Block. Häufiges Paar ist Write-Back mit Allocate und Write-Through mit No-Allocate. Die andere Diagonale ist zulässig. Nach einem Allocate-Fill im Write-Back-Modell ist die Line gültig und nach dem Store dirty. No-Allocate lässt den Cache-Eintrag unverändert und gibt den Store weiter. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 75 / 82

Software optimieren

- Hardware bekannt: Cache Lines (räumlich) und LRU (zeitlich)
- Gute Nachricht: Caches müssen in C nicht manuell programmiert werden
- Gegen das Modell zu programmieren kann Zugriffe verteuern. Ein fester Faktor wird nicht behauptet
- Ziel ist korrekter Code mit passender Lokalität, nicht maximale Trefferquote um jeden Preis

Notizen

Layout und Traversal beeinflussen die Lokalität. Normale cachebare Loads müssen nicht von Hand in den Cache gelegt werden. Nicht jeder Cache benutzt echtes LRU. Verstreute Zugriffe bleiben zulässig, wenn der Algorithmus sie braucht. Ohne Laufzeitmessung bleibt der Nutzen eine Aussage über das Modell. Korrektheit bleibt vor der Trefferquote. Ein zweidimensionales Array in C liegt zeilenweise. Der innere Index sollte deshalb die benachbarten Elemente besuchen, wenn das Modell räumliche Lokalität nutzen soll. MMIO und DMA bleiben außerhalb dieser normalen Load-Regel. volatile umgeht den Cache nicht. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 76 / 82

Labor-Szenario: 2D-Arrays

- Beispiel: Matrizen, Bilder, Tabellen
- Bild mit 1000×1000 Pixeln auf 0 setzen
- Variante A oder Variante B — mathematisch gleich, architektonisch nicht

```
int bild[1000][1000];

// Variante A oder Variante B?
```

Notizen

Labor-Szenario: 2D-Arrays schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Beispiel: Matrizen, Bilder, Tabellen. Bild mit 1000×1000 Pixeln auf 0 setzen. Variante A oder Variante B — mathematisch gleich, architektonisch nicht. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

2D-Arrays im RAM — Row-Major Order

- RAM ist linear, kein 2D-Gitter
- C speichert zeilenweise (Row-Major Order)
- `[0][0]`, `[0][1]`, `[0][2]` liegen nebeneinander
- `[0][0]` und `[1][0]` liegen 1000 Integers (≈ 4000 Bytes) auseinander

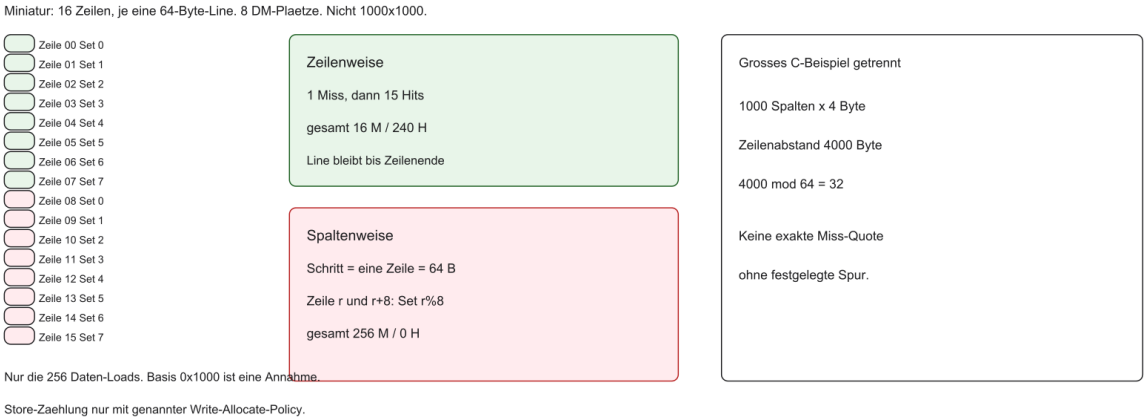


Abbildung 14: Miniaturmodell 16×16 getrennt von der 1000×1000-Matrix. Zeilenweise 16 Misses und 240 Hits, spaltenweise 256 Misses.

- C legt Zeilen hintereinander. Abstand `[0][0]` zu `[1][0]` bei 1000 Spalten und 4 Byte: 4000 Byte
- 4000 ist kein Vielfaches von 64; die erste Line einer Zeile beginnt nicht generell bei Spalte 0
- Die genannten Hit-/Miss-Zahlen gelten nur für das Miniaturmodell
- Stores der großen Matrix nur unter angenommener Write-Allocate-Policy; das sind nicht alle Zugriffe eines kompilierten Programms

Notizen

2D-Arrays im RAM — Row-Major Order schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. RAM ist linear, kein 2D-Gitter. C speichert zeilenweise (Row-Major Order). `[0][0]`, `[0][1]`, `[0][2]` liegen nebeneinander. `[0][0]` und `[1][0]` liegen 1000 Integers (≈ 4000 Bytes) auseinander. C legt Zeilen hintereinander. Abstand `[0][0]` zu `[1][0]` bei 1000 Spalten und 4 Byte: 4000 Byte. 4000 ist kein Vielfaches von 64; die erste Line einer Zeile beginnt nicht generell bei Spalte 0. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage,

bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 78 / 82

Cache-Albtraum: spaltenweiser Zugriff

```
// Column-Major Traversal - schlecht für den Cache
for (int col = 0; col < 1000; col++) {
    for (int row = 0; row < 1000; row++) {
        bild[row][col] = 0;
    }
}
```

Innere Schleife erhöht `row`: Zugriff `[0][0]`, `[1][0]`, `[2][0]`, ...

Notizen

Cache-Albtraum: spaltenweiser Zugriff schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Die Abbildung zeigt den nächsten Schritt des einfachen blockierenden Lehrmodells. Folgen Sie den Pfeilen von der Anfrage über die Prüfung bis zur Antwort. Zahlen in der Zeichnung sind Lehrannahmen. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 79 / 82

Was passiert beim spaltenweisen Zugriff?

- Im Miniaturmodell ist jeder Spaltenzugriff ein Miss
- Beim 1000×1000-Beispiel ist „nahe 100 %“ nur unter den genannten Annahmen eine Orientierung
- Die mitgeladenen Nachbarn einer Zeile werden bei Spaltentraversal nicht als Nächstes genutzt

Notizen

Im Miniaturmodell gibt es 16 Zeilen-Lines und nur acht Direct-Mapped-Plätze. Zeile r und Zeile r+8 teilen denselben Platz. Beim nächsten Spaltendurchlauf ist die zuerst gebrauchte Line schon verdrängt. Deshalb ergeben die 256 reinen Loads 256 Misses. Das große 1000-mal-1000-Beispiel hat keine exakte Quote

ohne festgelegte Spur. Adresse ist 0x1000 plus 4 mal (16 mal Zeile plus Spalte). Zeile und Zeile plus 8 teilen Set Zeile modulo 8. Der zweite Spaltendurchlauf findet die erste Line nicht mehr. Für Stores unter Write-Back und Write-Allocate kommen Dirty-Writebacks hinzu. Die Miss-Zahl ist dann nicht die Writeback-Zahl. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 80 / 82

Cache-Traum: zeilenweiser Zugriff

Nur die Schleifen vertauschen:

```
// Row-Major Traversal - cache-freundlich
for (int row = 0; row < 1000; row++) {
    for (int col = 0; col < 1000; col++) {
        bild[row][col] = 0;
    }
}
```

- Unter Write-Allocate und gültig bleibender Line folgen auf den ersten Store einer Line weitere Hits
- Der Compiler kann Schleifen entfernen, vertauschen oder in Bibliotheksaufrufe umwandeln
- Ein Geschwindigkeitsfaktor ist damit nicht allgemein festgelegt

Notizen

Die innere Schleife läuft über die Spalten einer Zeile. Im Miniaturmodell passen 16 Werte einer ausgerichteten Zeile in eine 64-Byte-Line: ein Miss und 15 Hits. Über 16 Zeilen sind das 16 Misses und 240 Hits. Das gilt für die definierte Load-Spur. Ein Compiler kann die Schleife verändern. Es ist kein gemessener Speedup. Pro ausgerichteter Zeile: der erste Load ist ein Miss, die nächsten 15 Loads treffen dieselbe gültige Line. 16 Zeilen ergeben 16 Misses und 240 Hits, Miss Rate 6,25 Prozent. Dieselbe Zählung für Stores gilt nur bei Write-Allocate. memset oder eine entfernte Schleife wären eine andere Spur. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 81 / 82

Ausblick: Visualisierung im Labor

- Ripes kann Hit-, Miss- und Writeback-Zähler einer Zugriffsspur zeigen
- Der Cache-Simulator modelliert laut Dokumentation keine Cache-Latenzen und damit keine CPU-Laufzeit
- Die Memory-Ansicht zeigt auch im Write-Back-Modus die geschriebenen Werte; sie beweist keinen veralteten DRAM-Inhalt
- Für die Zählung eignet sich das Single-Cycle-Modell; Pipeline-Stalls können zusätzliche Zugriffe zählen
- Vergleiche nur bei gleicher Kapazität, Line-Größe, Startzustand und Spur. Ein Exponent in der UI ist keine Anzahl

Notizen

Ausblick: Visualisierung im Labor schließt an die bisherige Cache-Frage an. Ein Cache hält ausgewählte Blockkopien und verkürzt häufige Zugriffe im Mittel. Ripes kann Hit-, Miss- und Writeback-Zähler einer Zugriffsspur zeigen. Der Cache-Simulator modelliert laut Dokumentation keine Cache-Latenzen und damit keine CPU-Laufzeit. Die Memory-Ansicht zeigt auch im Write-Back-Modus die geschriebenen Werte; sie beweist keinen veralteten DRAM-Inhalt. Für die Zählung eignet sich das Single-Cycle-Modell; Pipeline-Stalls können zusätzliche Zugriffe zählen. Vergleiche nur bei gleicher Kapazität, Line-Größe, Startzustand und Spur. Ein Exponent in der UI ist keine Anzahl. Jede Zahl gilt mit ihrer Einheit und der genannten Annahme. Eine C-Zeile ist ohne Übersetzung keine beobachtete Maschinenspur. Hit Rate zählt Treffer einer betrachteten Folge. Die nächste Folie nimmt die noch offene Frage auf. Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.

Folie 82 / 82

Zusammenfassung Block 2

- Memory Wall erzwingt Speicherhierarchie
- Caches basieren auf Lokalität (zeitlich und räumlich)
- AMAT = Hit Time + Miss Rate $\times$ Miss Penalty
- Direct Mapped: schnell, aber Thrashing-anfällig
- Set-Associative: Kollisionen innerhalb eines Sets entschärfen
- Write-Through und Write-Back bestimmen die Weitergabe geänderter Cache-Daten
- Store-Miss: Write-Allocate oder No-Write-Allocate, unabhängig davon
- Row-Major nutzt Lines; das Miniaturmodell macht die Zählung explizit

Notizen

Die Hierarchie nutzt Lokalität. Mapping legt den Platz fest, das Tag identifiziert den Block, Replacement wählt das Opfer, die Schreibpolicy die Weitergabe. AMAT ist das modellierte Mittel, nicht der Worst Case. Write-Through und Write-Back sind unabhängig von Write-Allocate und No-Write-Allocate. Prüfen Sie: Muss ein L1-Miss DRAM erreichen? Darf eine Dirty-Line verworfen werden? Warum unterscheiden sich die Miniatur-Traversals? Halten Sie drei Ebenen auseinander: die Speicheranfrage, die Cache-Verwaltung und das C-Quellprogramm. Eine Zahl gilt nur mit ihrer Einheit und der genannten Annahme. Sie ist keine Messung und kein Laufzeitergebnis. Steht Code auf der Folie, ist er Lehrcode. Eine RISC-V-Toolchain, Renode und ein Kohärenz-Zeitmodell waren nicht verfügbar, deshalb wurde er nicht übersetzt und nicht ausgeführt. Prüfen Sie danach die offene Frage, bevor Sie die nächste Folie nur als Wiederholung lesen.